ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Internet Data Center

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria

michele.colajanni@unibo.it

Replication is the solution for reliability and scalability

Replication → **System bottlenecks**, **Single points of failure**

- **Replication of server machines**
- **Replication of interconnections**
- **Replication of data**
- **Replication of application processes**
- **Replication of people**

System Engineers — ***Infrastructure***

Computer Engineers

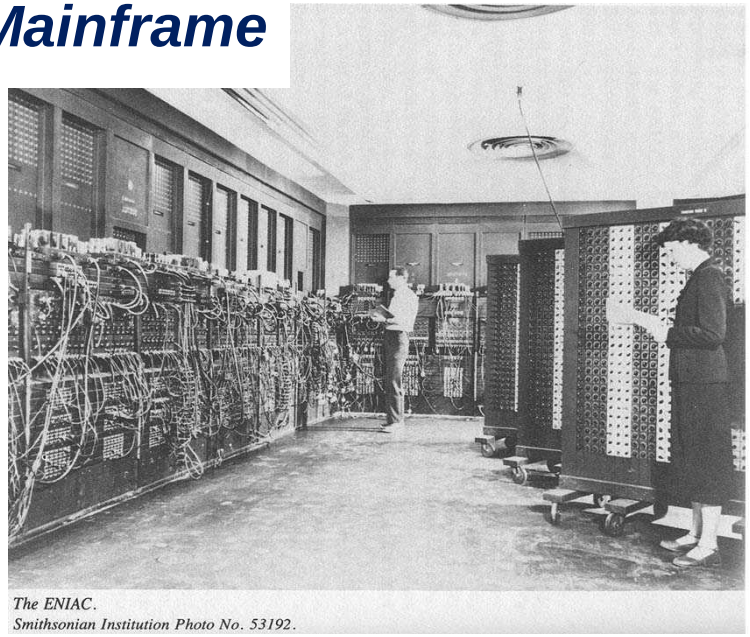
Managers *(one day…)*

IDC is the back-end of any S&R modern service

An Internet Data Center (IDC) is the back-end infrastructure of any large company to provide scalable (elastic), reliable, secure services delivered through the Internet

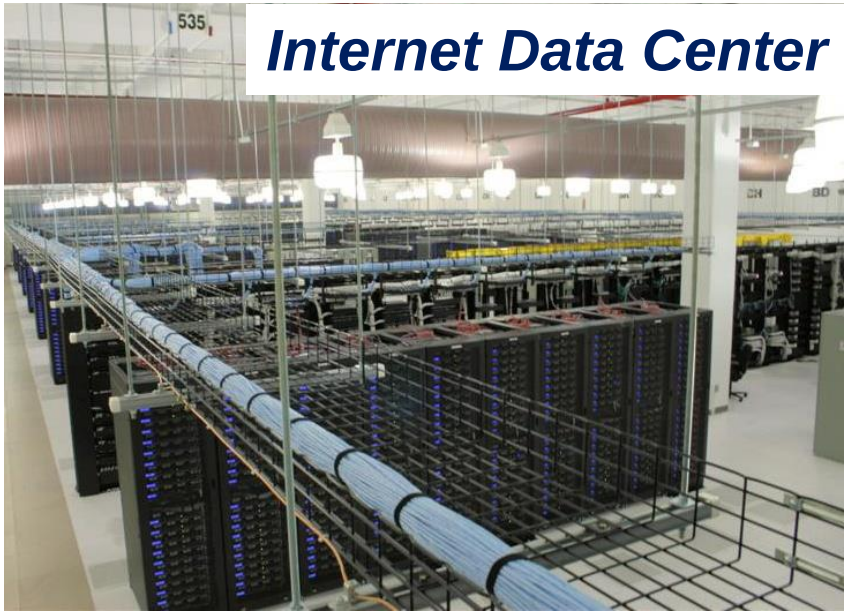
The data center comes from the past with a difference

Mainframe

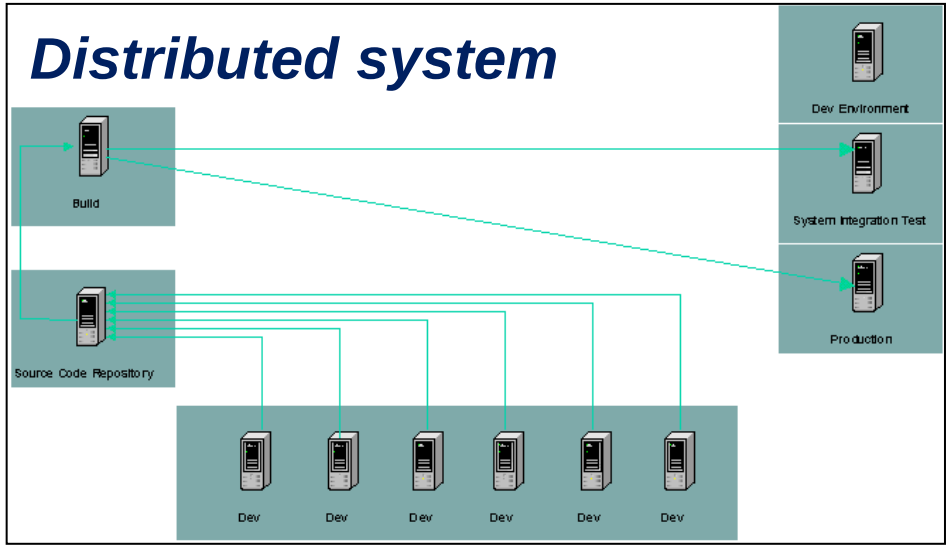


The ENIAC.
Smithsonian Institution Photo No. 53192.

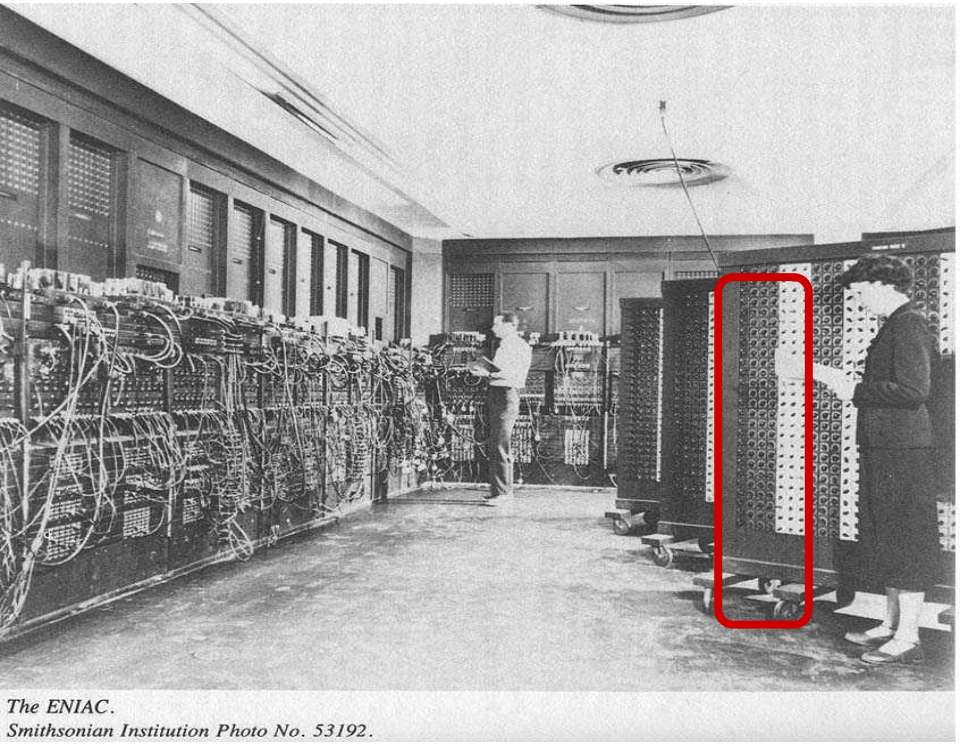
Internet Data Center



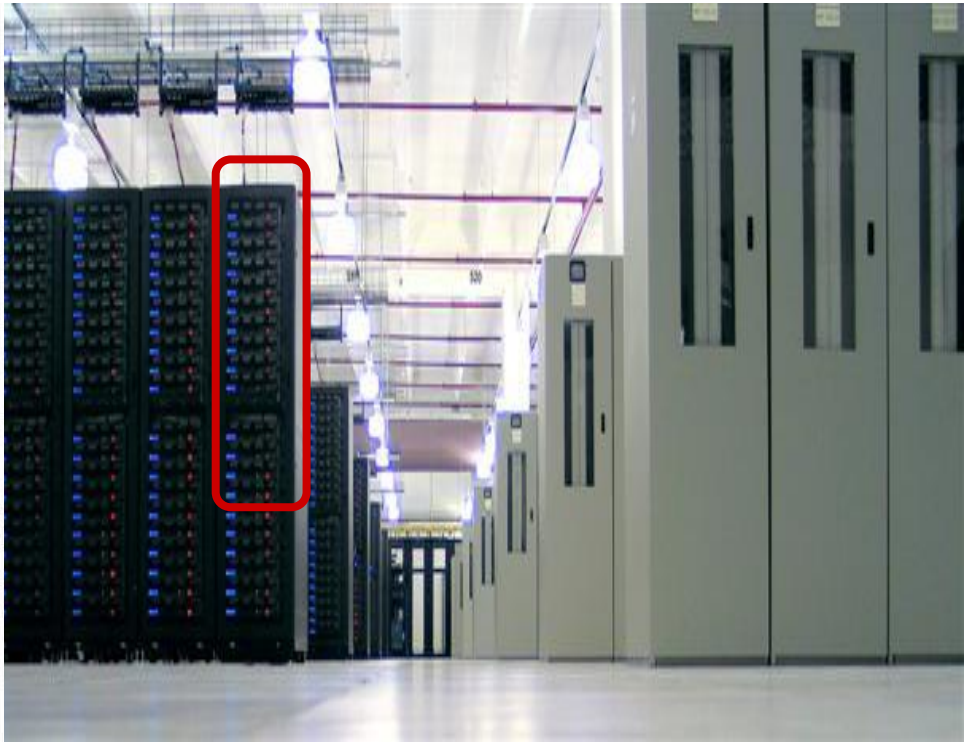
Distributed system



Similar, but with two main differences



The ENIAC.
Smithsonian Institution Photo No. 53192.



ENIAC

Internet Data Center

Differences?

1) Internet
2) Composition of thousands of server machines

The two visions

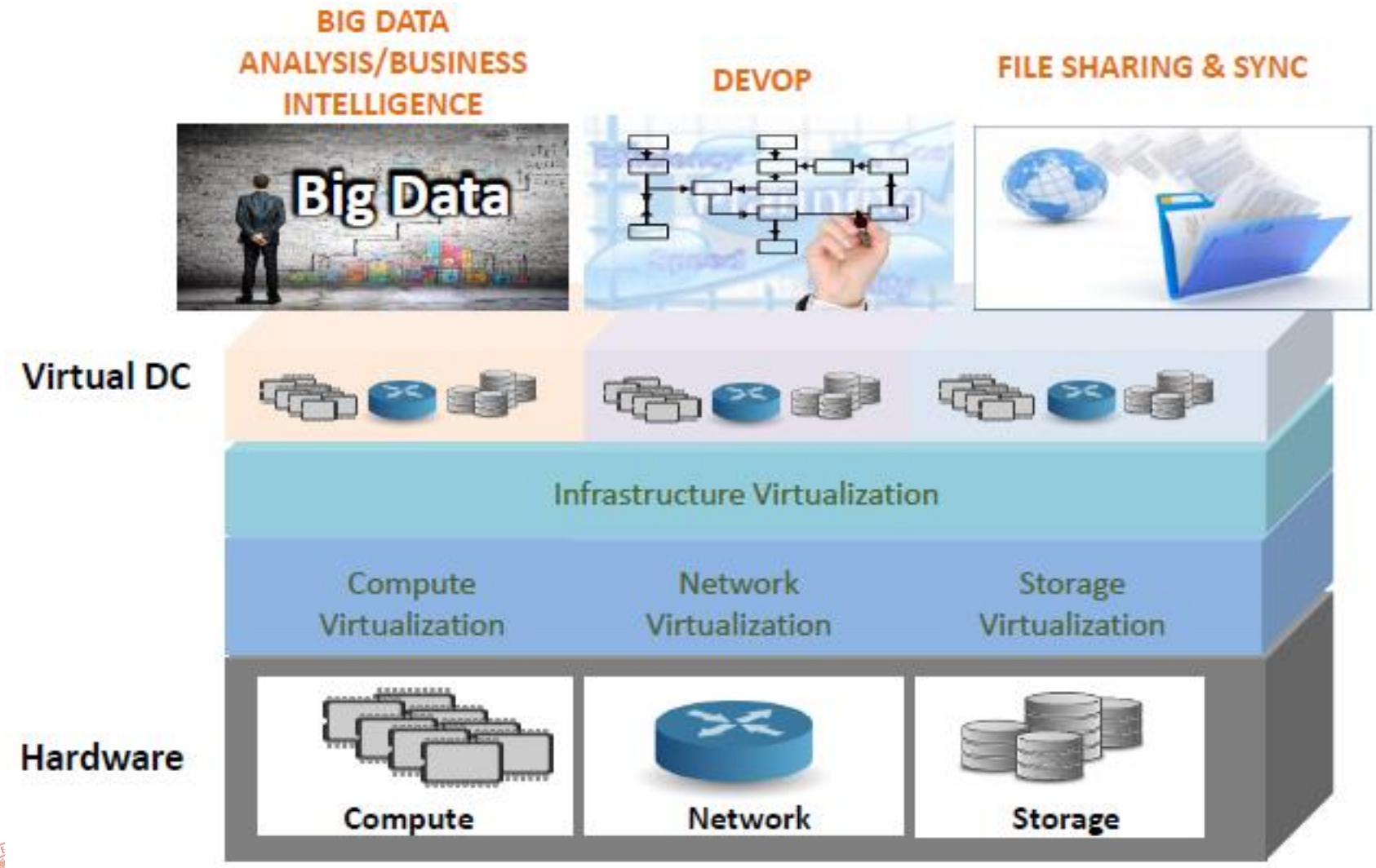
Offered vision (*User's vision*) ➔ **A logically isolated set of resources that are elastic (*scalable in two directions*), reliable, secure**

Internal vision (*Engineering vision*) ➔ **A large-scale, flexible and modular physical infrastructures consisting of**

- Racks
- PODs
- Storage systems
- Virtualization
- Advanced internal networking
- Advanced external networking

and a lot of other support infrastructures (monitoring, power, cooling, physical security, logistics, …)

The overall picture



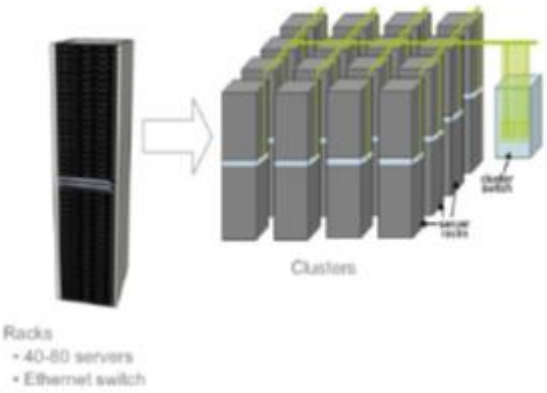
Server components

Basic ingredients for a Datacenter

Blade servers for computation

Rack

Interconnected racks



Basic ingredients for a Datacenter

Storage

The basics

- Disk trays
- SSD & NVM Flash

NVMe Flash

JBOD disk array

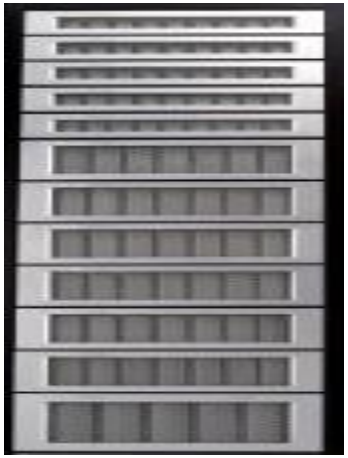
What's new

- Non-volatile memories
- New archival storage (e.g., glass)

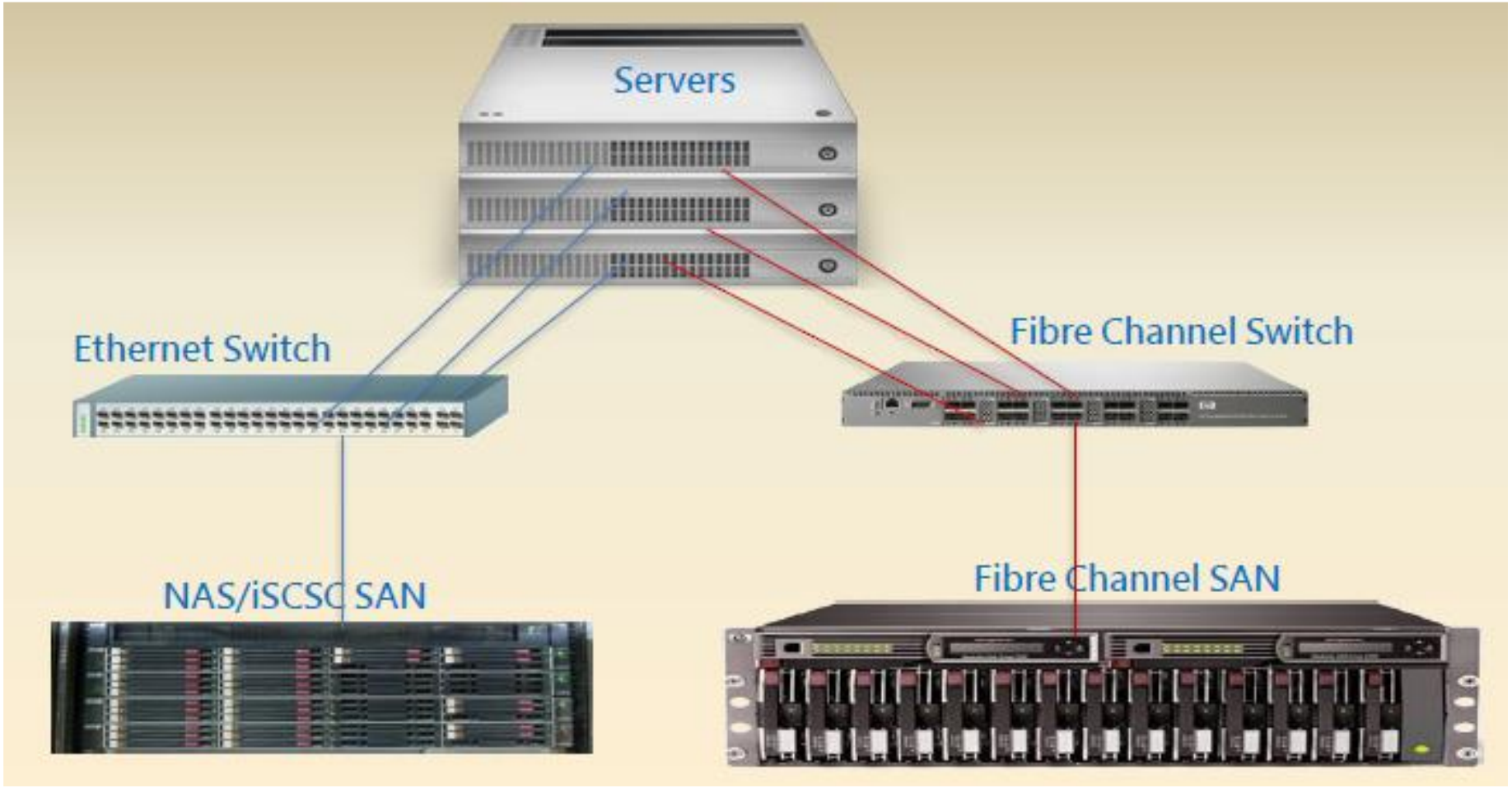
NVM DIMM

Distributed with compute or NAS systems

Blade servers for storage



Typical connections

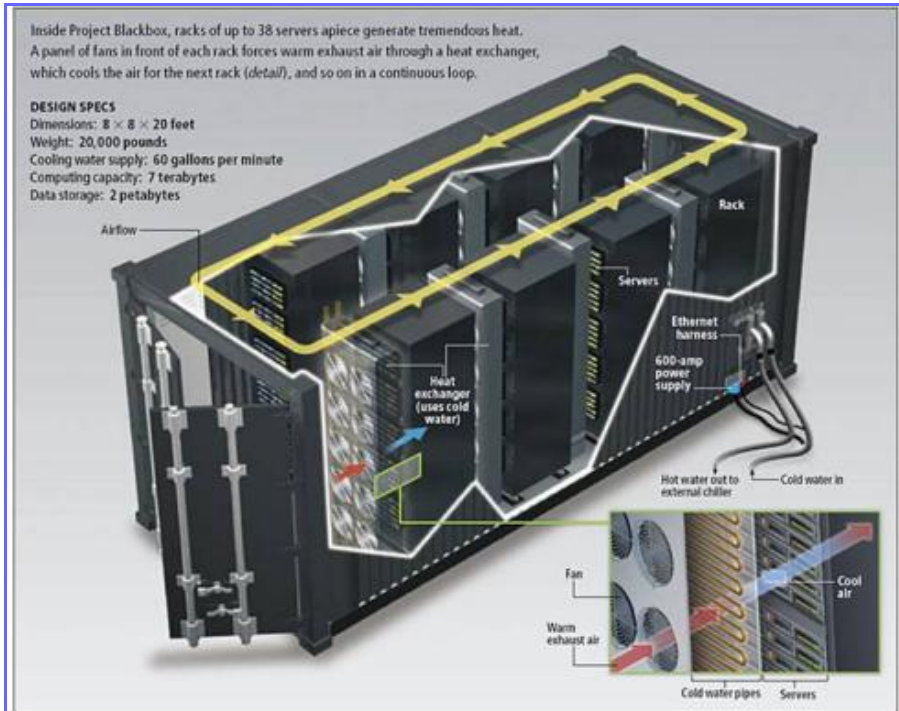


Typical datacenter

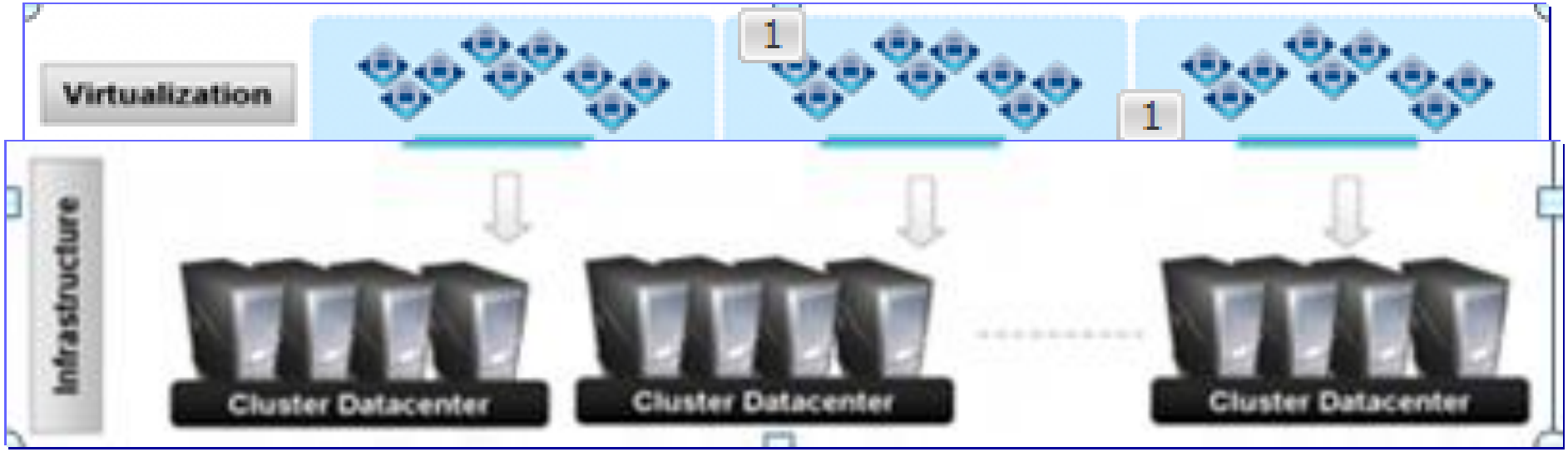
Other types: *POD*

POD = Performance Optimized Datacenter

- A container-type data center, including multiple racks (10-20), data storage devices, network infrastructure, power plant, cooling system
- Modular approach to data center expansion that uses preconfigured components to provide extra power capacity where it is needed



Virtualization is the fundamental ingredient

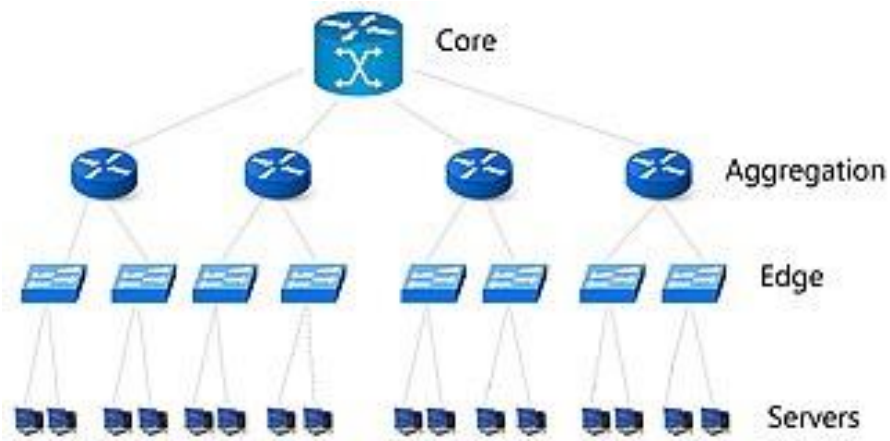


Internal network

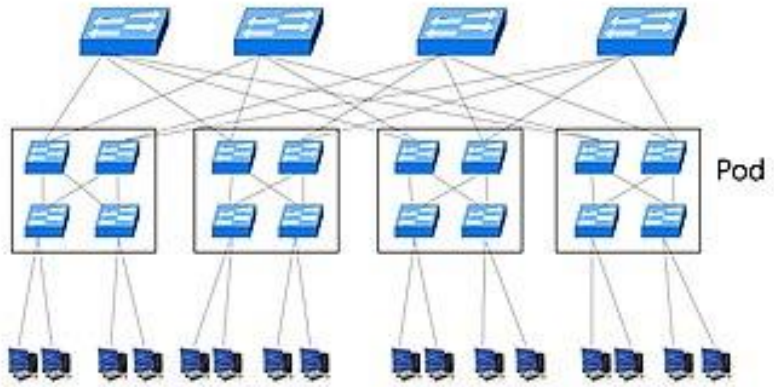
The advantages of the *internal* network

- **The internal networks of data centers are not mini-Internets**
- **Facilitators:**
  - **single owner/administration domain** [*likely, the most important difference with GRID computing*]
  - **limited hardware/software diversity**
  - **no malicious or selfish node**
  - **cloud providers know (and can define) the best topology in terms of performance, availability, scalability**

Data Center networking (1st generation)

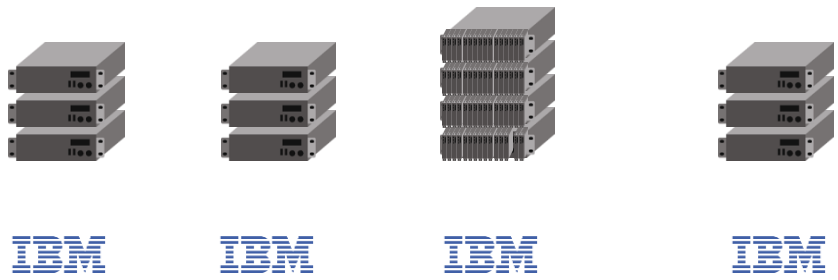


Basic Tree

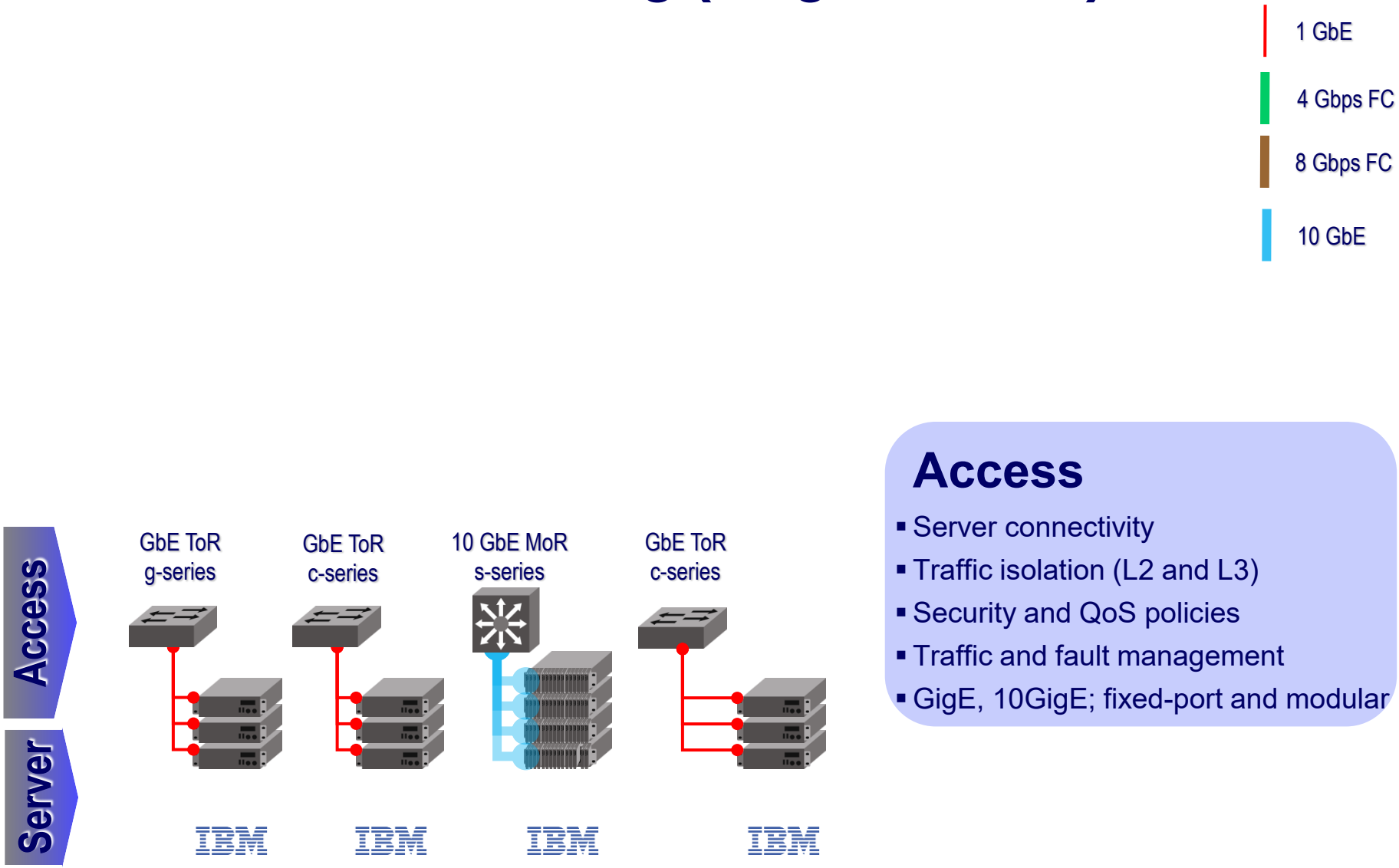


Fat Tree

Data Center networking (1st generation)



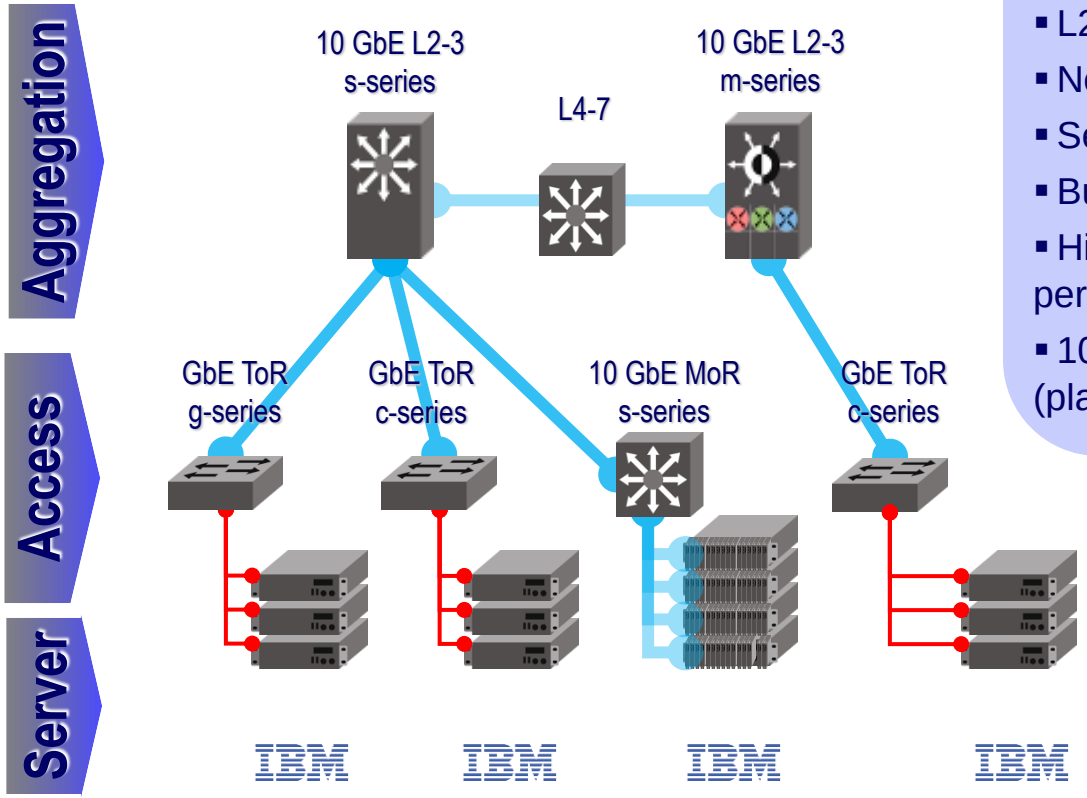
Data Center networking (1st generation)



Access

- Server connectivity
- Traffic isolation (L2 and L3)
- Security and QoS policies
- Traffic and fault management
- GigE, 10GigE; fixed-port and modular

Data Center networking (1st generation)

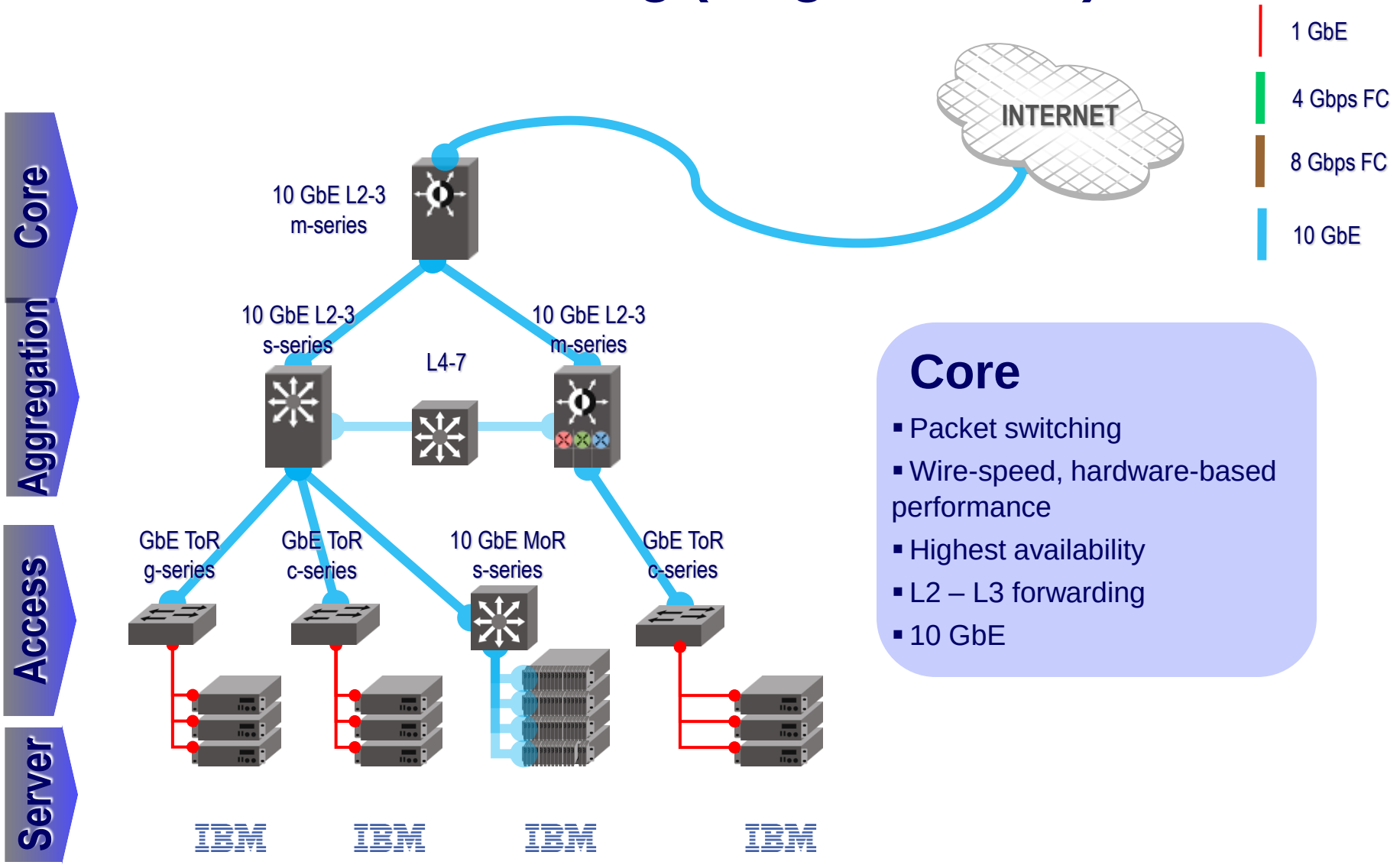


Aggregation

- Traffic aggregation
- L2 to L3 transition
- Network visibility
- Security
- Bundle QoS
- High density, availability, performance
- 10 GbE; optional L3 isolation (platform)

1 GbE
4 Gbps FC
8 Gbps FC
10 GbE

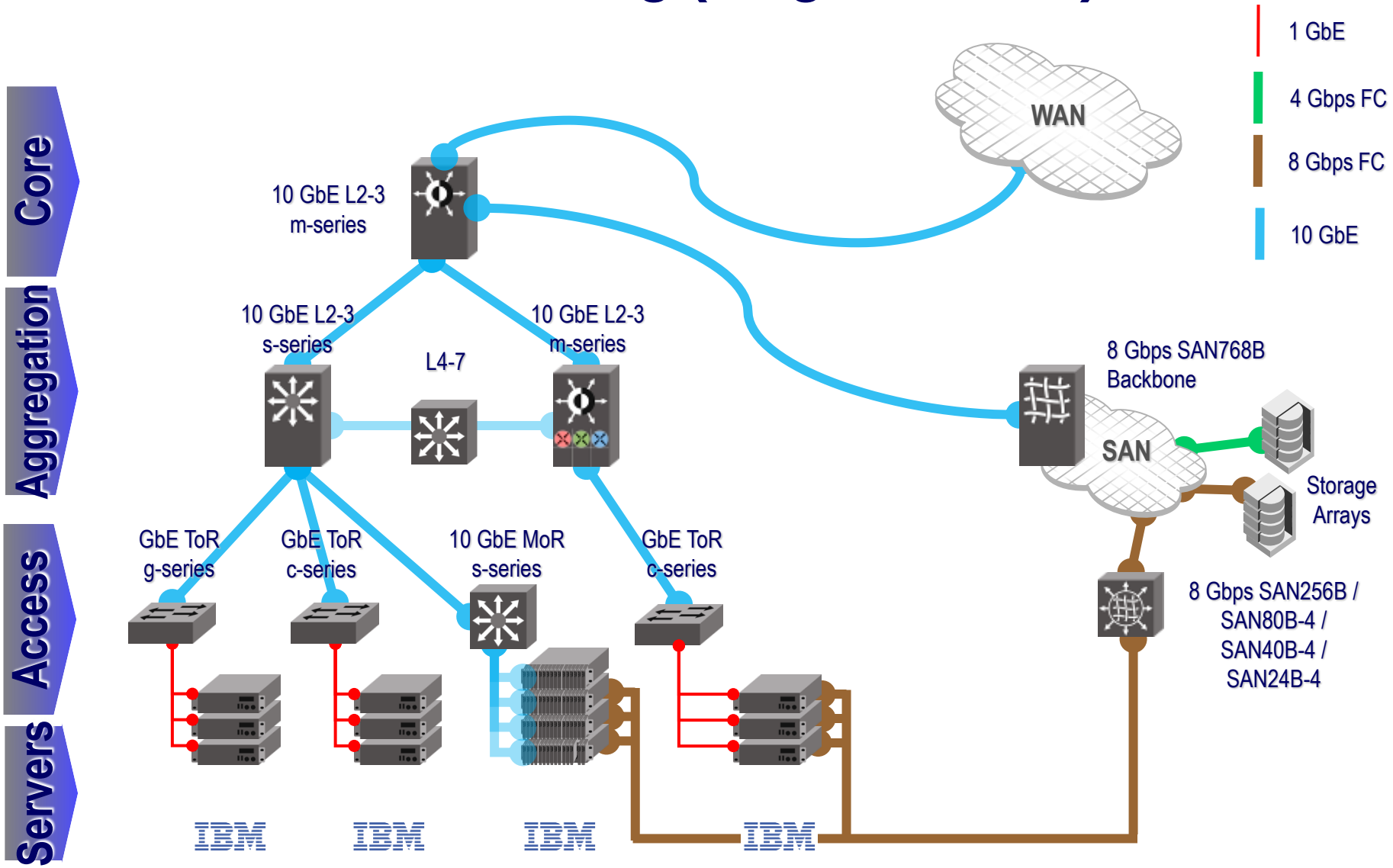
Data Center networking (1st generation)



Core

- Packet switching
- Wire-speed, hardware-based performance
- Highest availability
- L2 – L3 forwarding
- 10 GbE

Data Center networking (1st generation)



MAIN GOAL for modern datacenter → *Agility*

- **Agility inside one datacenter → any virtual server can be dynamically assigned to any service anywhere in the data center**
- Unfortunately, conventional datacenter network designs work against agility by ***fragmenting both network and server capacity***, and limiting ***elasticity*** (=the dynamic growing and shrinking of the server pools)
- Multiple applications run inside a single datacenter, typically with each application hosted on its own set of (potentially virtual) server machines

Datacenter traffic

A datacenter network supports two types of traffic:

a) **Traffic flowing between external systems and internal servers**. For example, in *video download* applications, external traffic dominates

b) **Traffic flowing among internal servers**. For example, in *search applications* and *customized responses*, internal traffic dominates because of building and synchronizing instances of the indexes, and inter-process communications

In-Out traffic

- To support external requests from the Internet, an application is associated with one or more publicly visible and routable IP address(es) to which clients send their requests and from which they receive replies
- Inside the Data Center, the requests are spread by a ***specialized hardware load balancer*** among a pool of front-end servers that process the requests
- The IP address to which requests are sent is called a **virtual IP address** (**VIP**)
- The IP addresses of the servers over which the requests are spread are known as **direct IP addresses** (**DIP**)

Solutions for agility

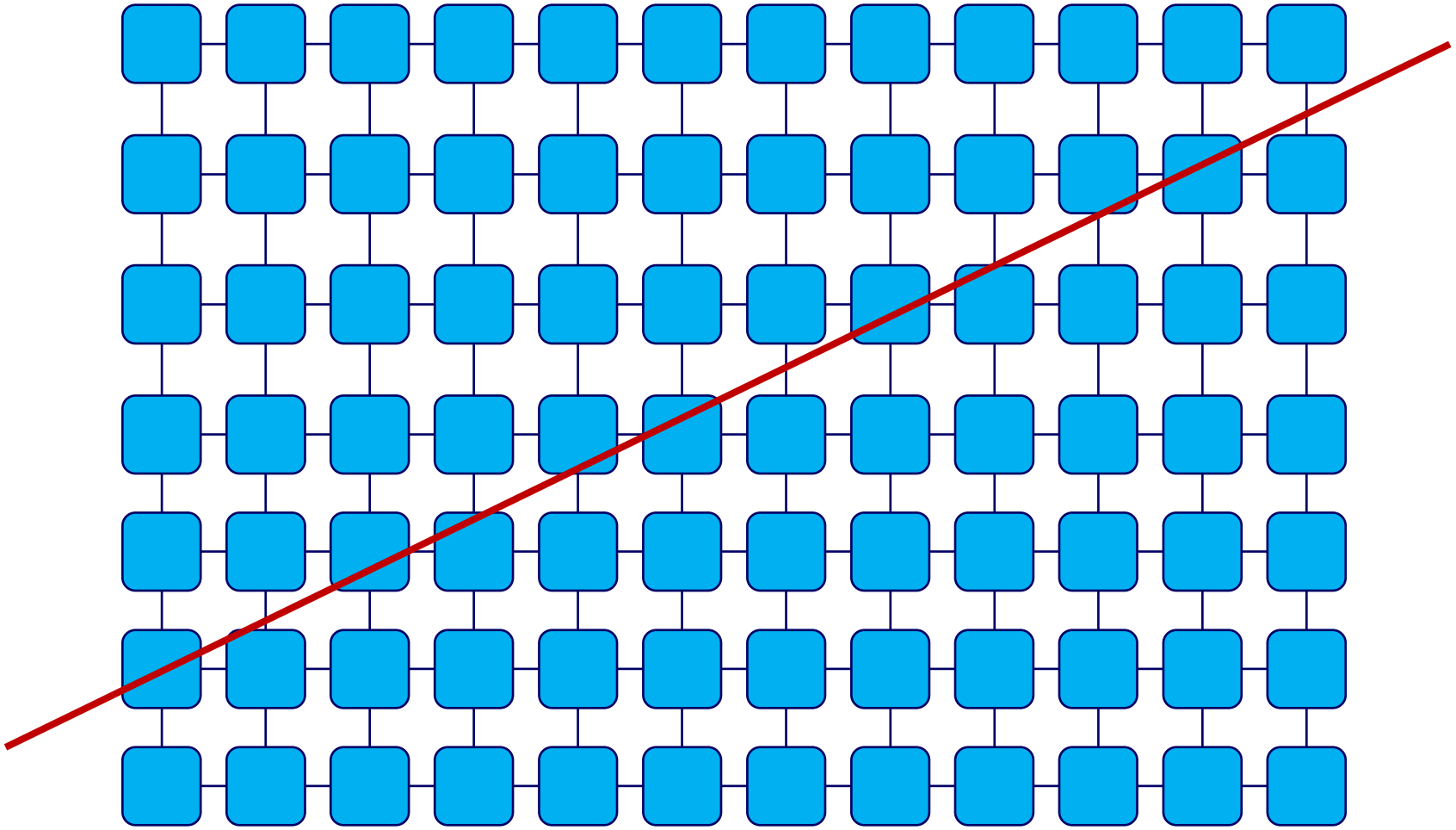
Location-independent addressing

- Services should use location-independent addresses that decouple the server's location in the datacenter from its address. This could enable any server to become part of any server pool while simplifying configuration management

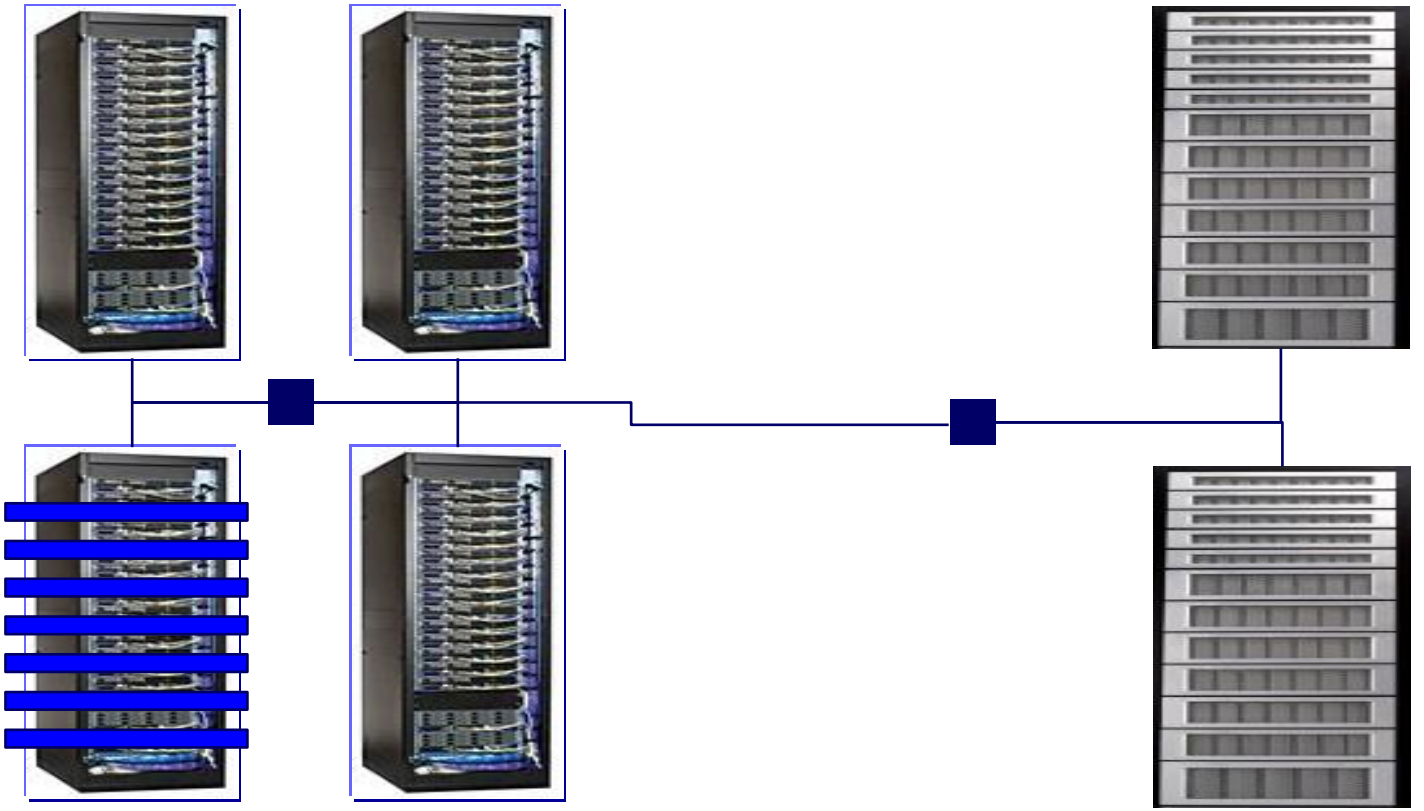
Uniform bandwidth and latency

- If the available bandwidth between two servers is not dependent on where they are located, then the servers for a given service could be distributed arbitrarily in the data center without fear of running into bandwidth bottleneck points
- Uniform bandwidth, combined with uniform latency between any two servers would allow services to achieve same performance regardless of the location of their servers

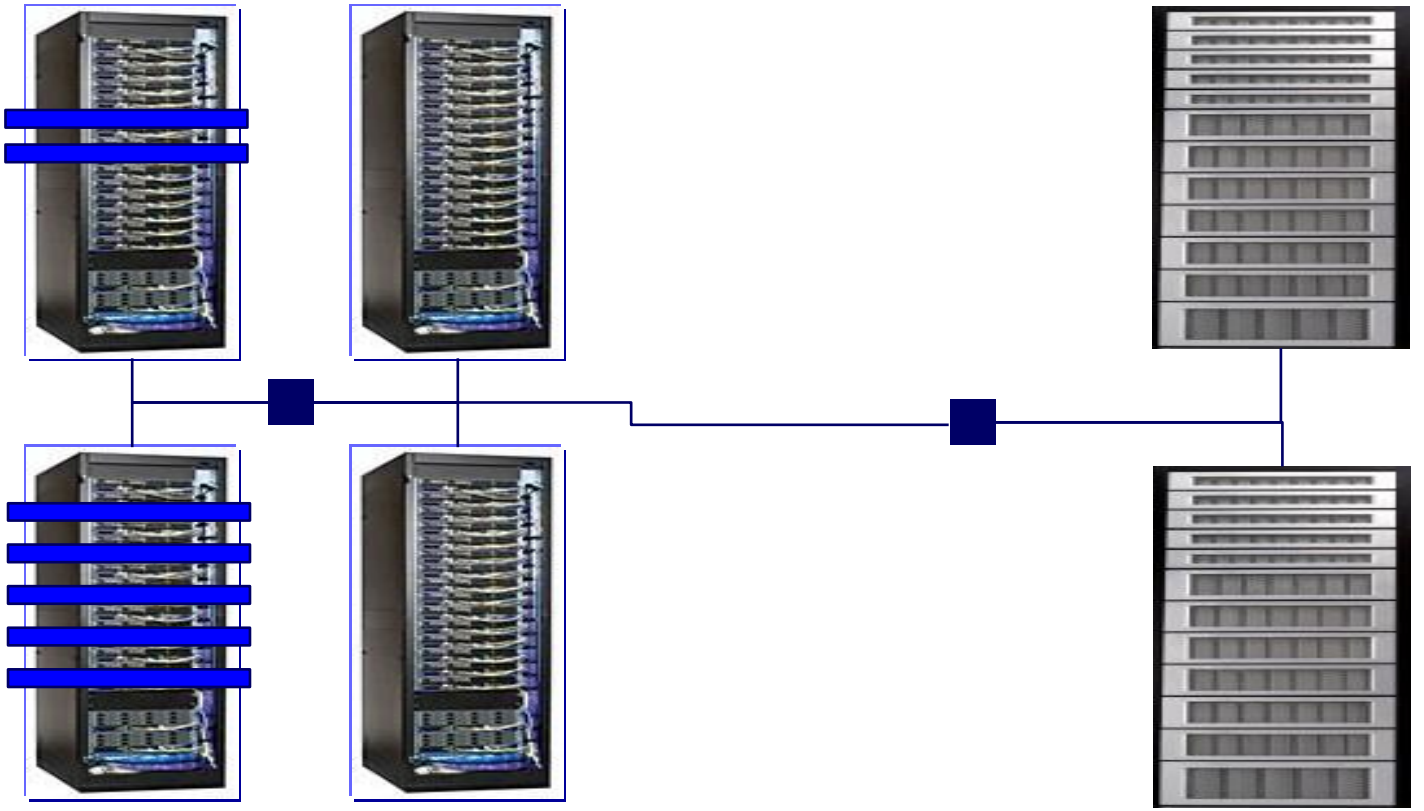
Server resources in a datacenter based on switches and routers do not represent a uniform pool



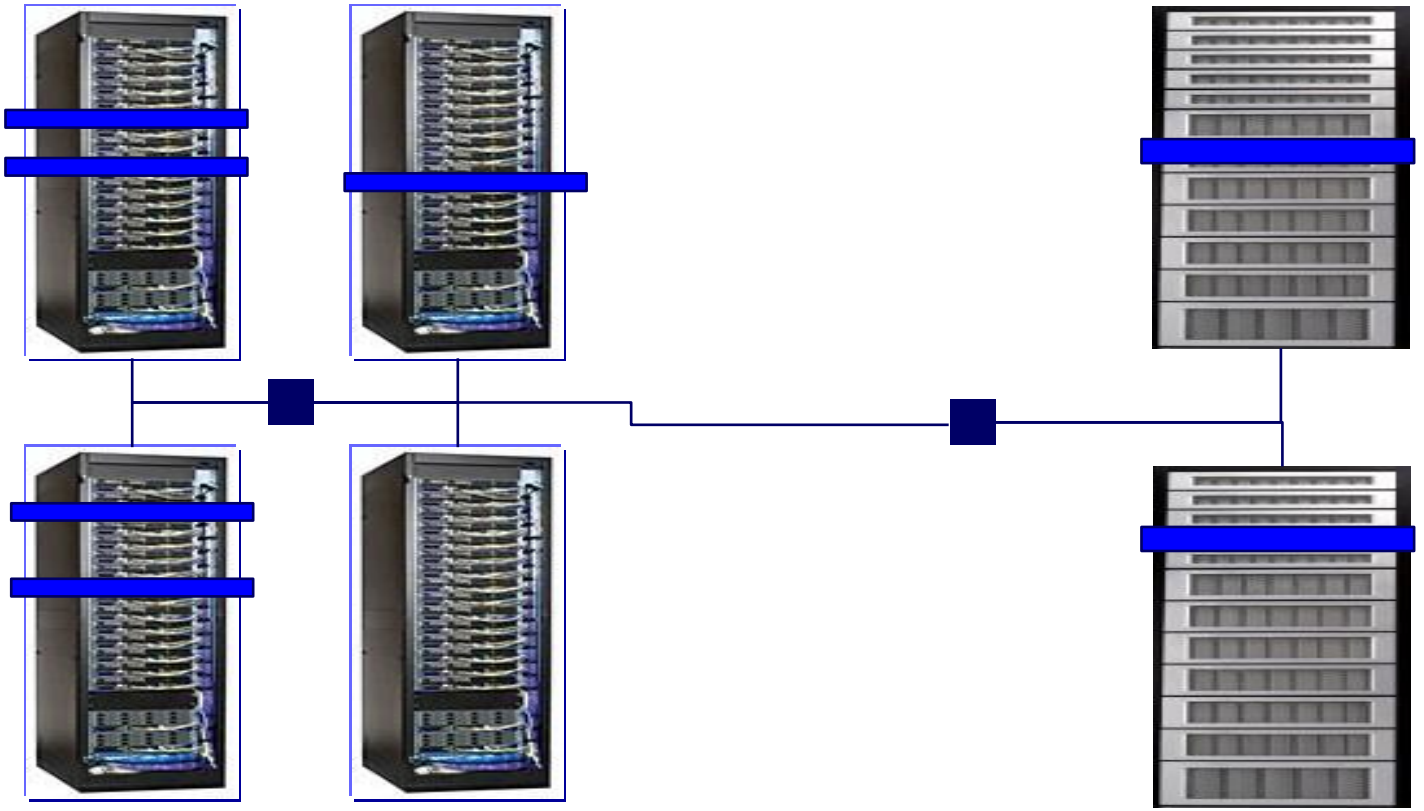
Your application at time T0



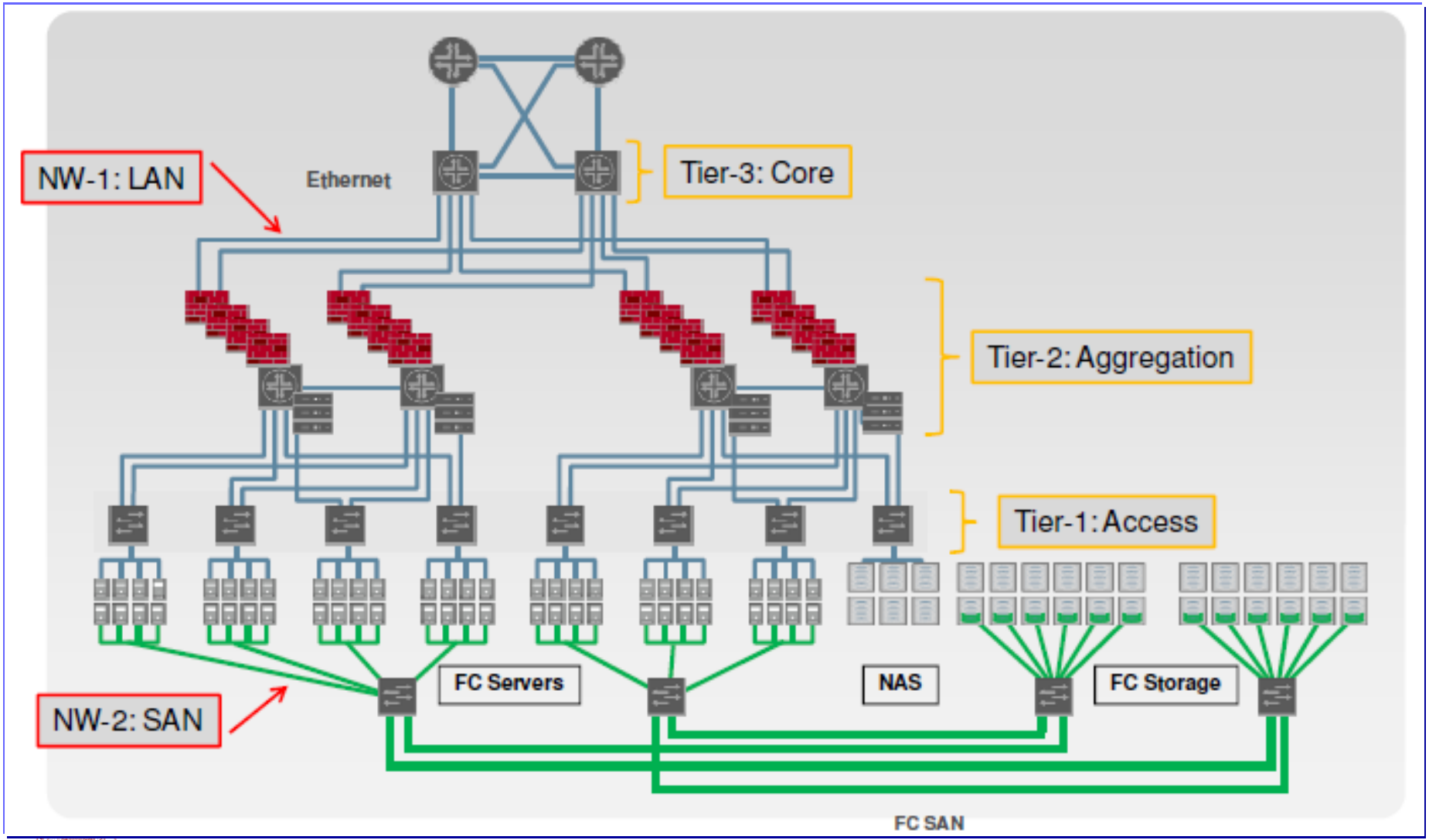
Your application at time T1



Your application at time T2



Internal network



Problem 1: *Static network assignment*

- To support internal traffic within the datacenter, individual applications are mapped to specific physical switches and routers, relying heavily on VLANs and layer-3 based VLAN spanning to cover the servers dedicated to the application
- Positive consequences:
  - High degree of performance and security isolation
- Negative consequences:
  - VLANs are often **policy-overloaded**, integrating traffic management, security, and performance isolation
  - VLAN and large server pools **concentrate traffic on links high in the tree**, where links and routers are highly overbooked

Problem 2: Fragmentation of resources

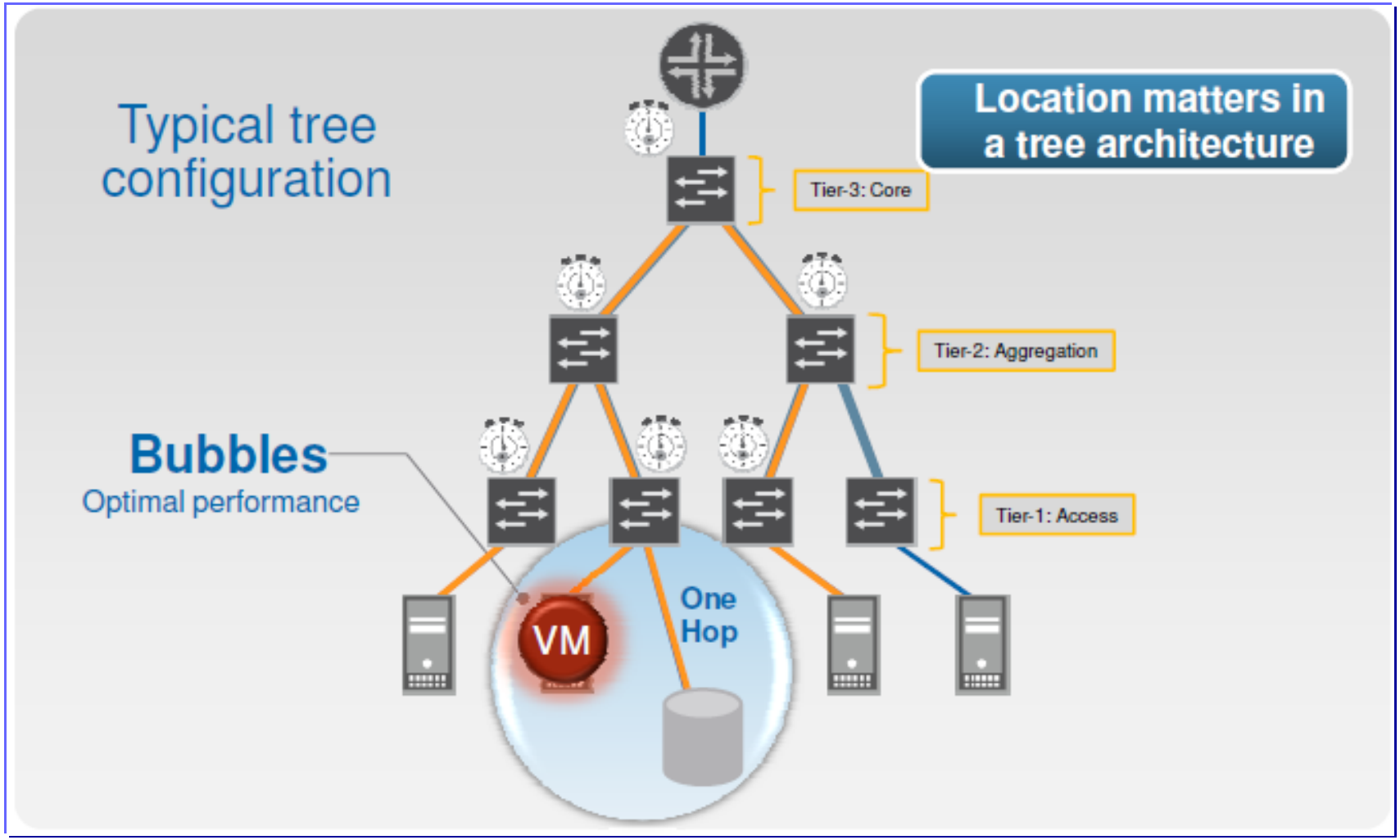
- **Some load balancing techniques (half NAT)** require that all IPs in a pool of Virtual IP be in the same layer 2 domain
  - PROBLEM: If an application grows and requires more front-end servers, it cannot use available servers in other layer 2 domains - ultimately resulting in fragmentation and under-utilization of resources
- **Load balancing via full NAT** allows servers to be spread across multiple layer 2 domains
  - PROBLEM: the servers do not see the client IP, which is unacceptable because servers use the client IP for everything, from data mining and response customization to regulatory compliance

Problem 3: Poor server-to-server connectivity

The communication between servers in different layer 2 domains must go through the layer 3 network portions

- Layer 3 ports are significantly more expensive than layer 2 ports (cost of supporting large buffers, marketplace factors). As a result, these links are typically oversubscribed by factors of 10:1 to 80:1 ➔ *The bandwidth available between servers in different parts of the datacenter can be limited*
- *Managing the scarce internal bandwidth is an interesting global optimization problem*: servers from all applications must be placed with great care to ensure that the sum of their traffic does not saturate any of the network links → Unfortunately, achieving this level of coordination between continuously changing applications is impossible in practice

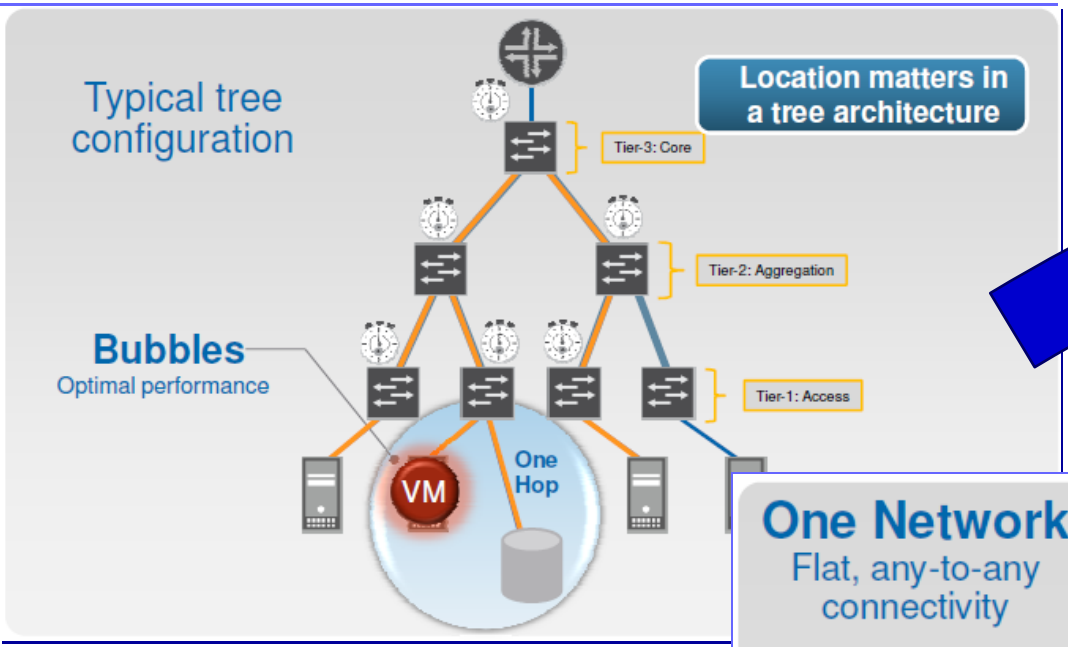
Location matters (and this is not good for performance)

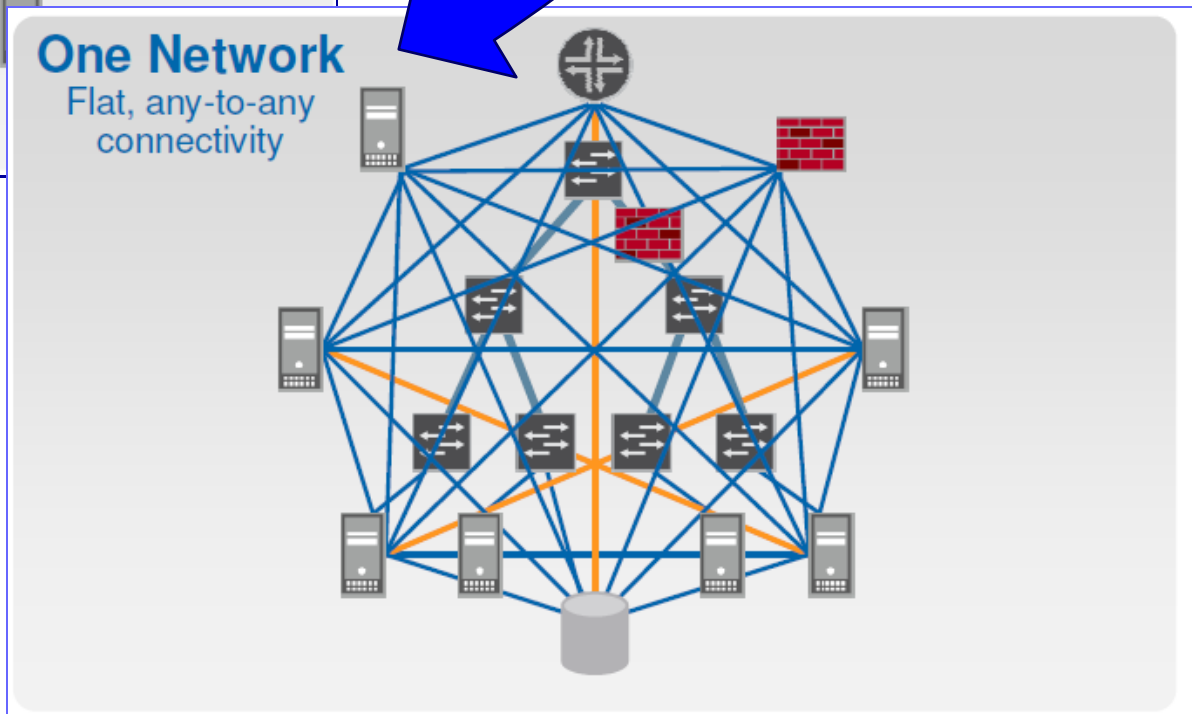


Technical solutions for agility

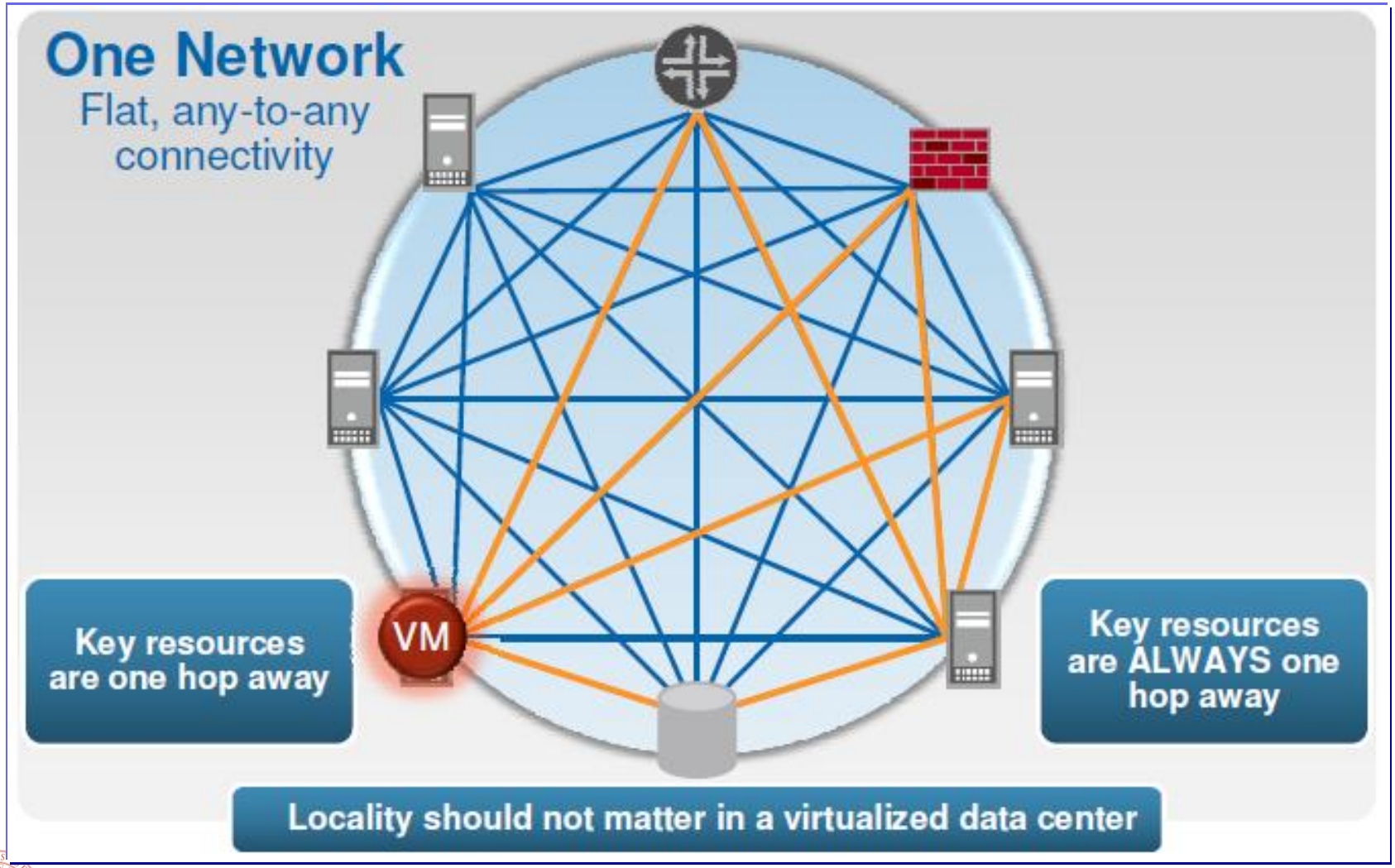
- A number of approaches are being explored to meet the requirements of *agility* in intra datacenter networks
  - **Location-independent addressing**
  - **Uniform bandwidth and latency**
- Major commercial vendors (e.g., Cisco, Juniper) are developing **Data Center Ethernet** or **One Network**, which uses layer 2 addresses for location independence and complex congestion control mechanisms

Target to reach

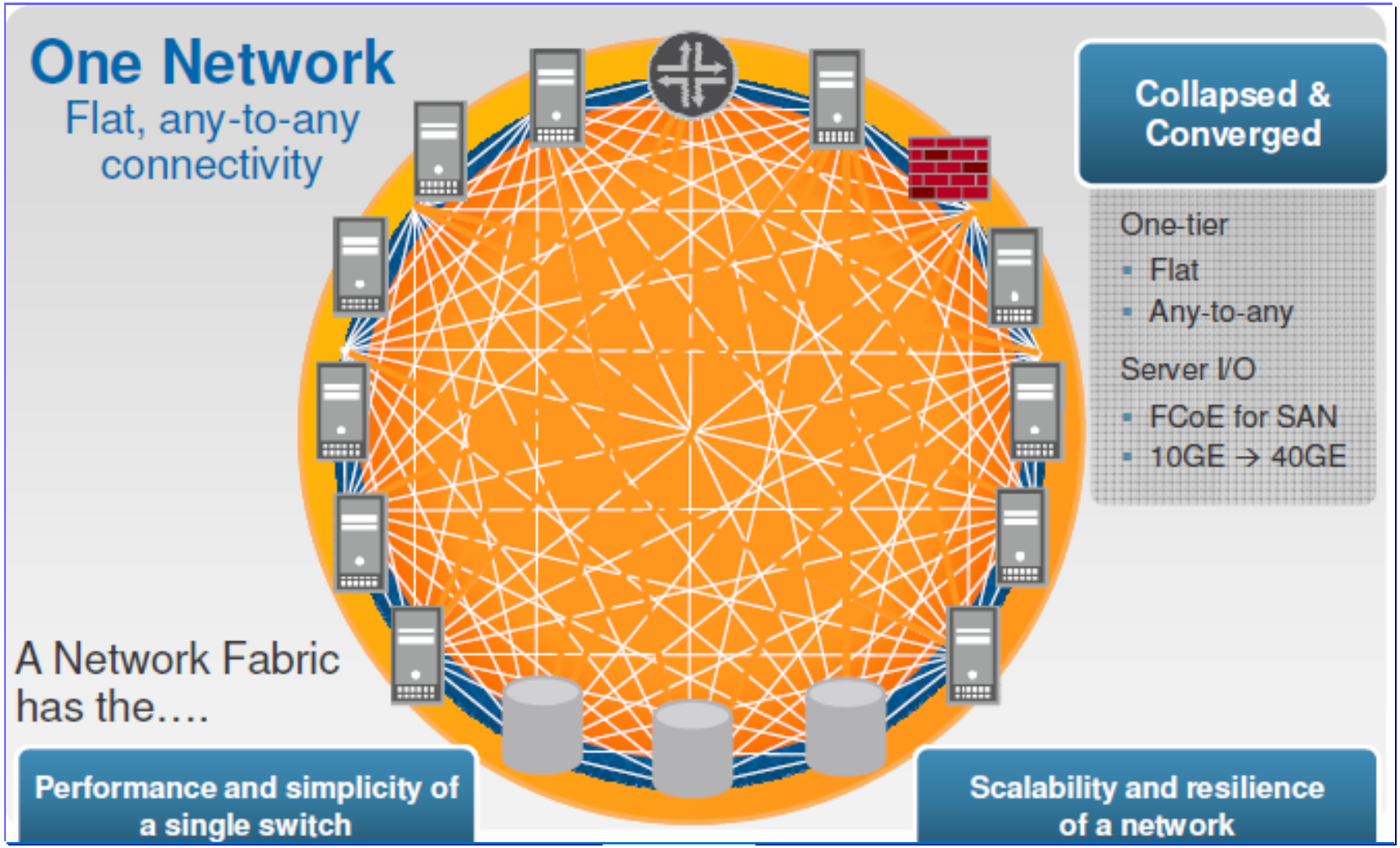




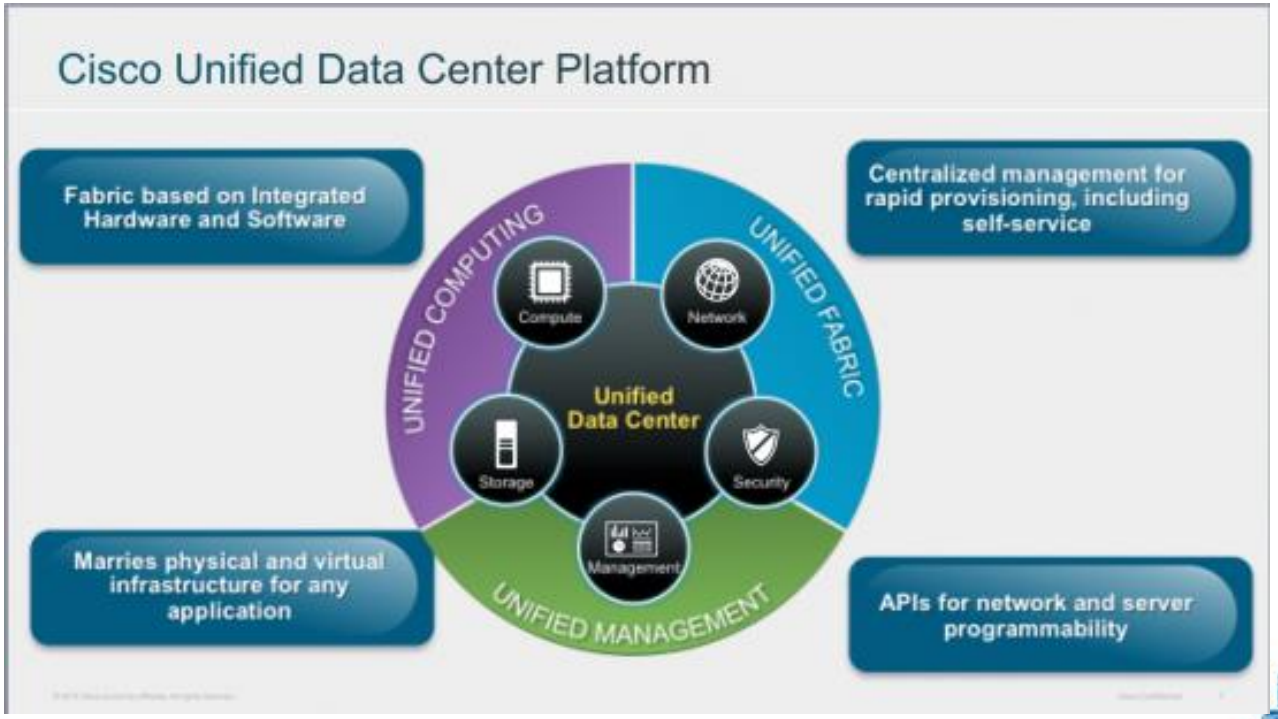
Locality should not matter

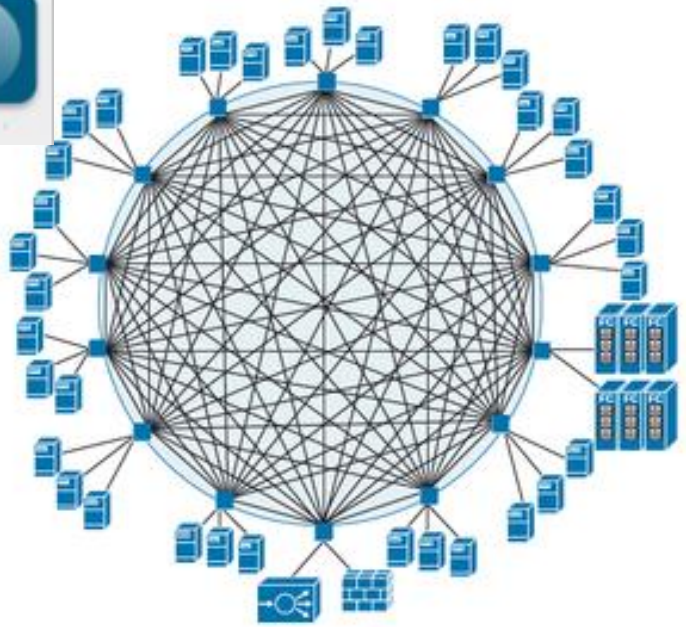


Datacenter Fabric (e.g., Juniper)



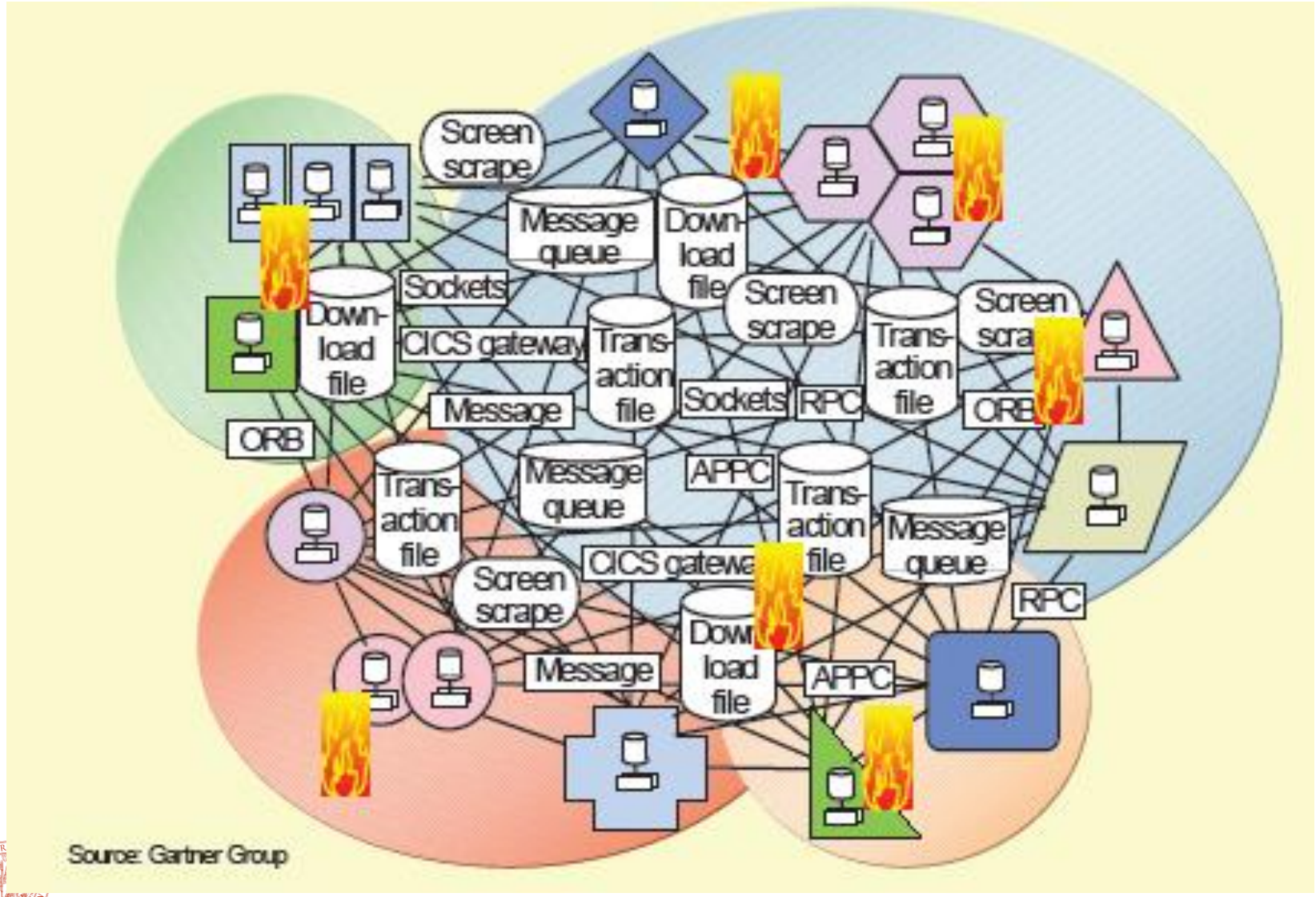
Unified Data Center Fabric by Cisco



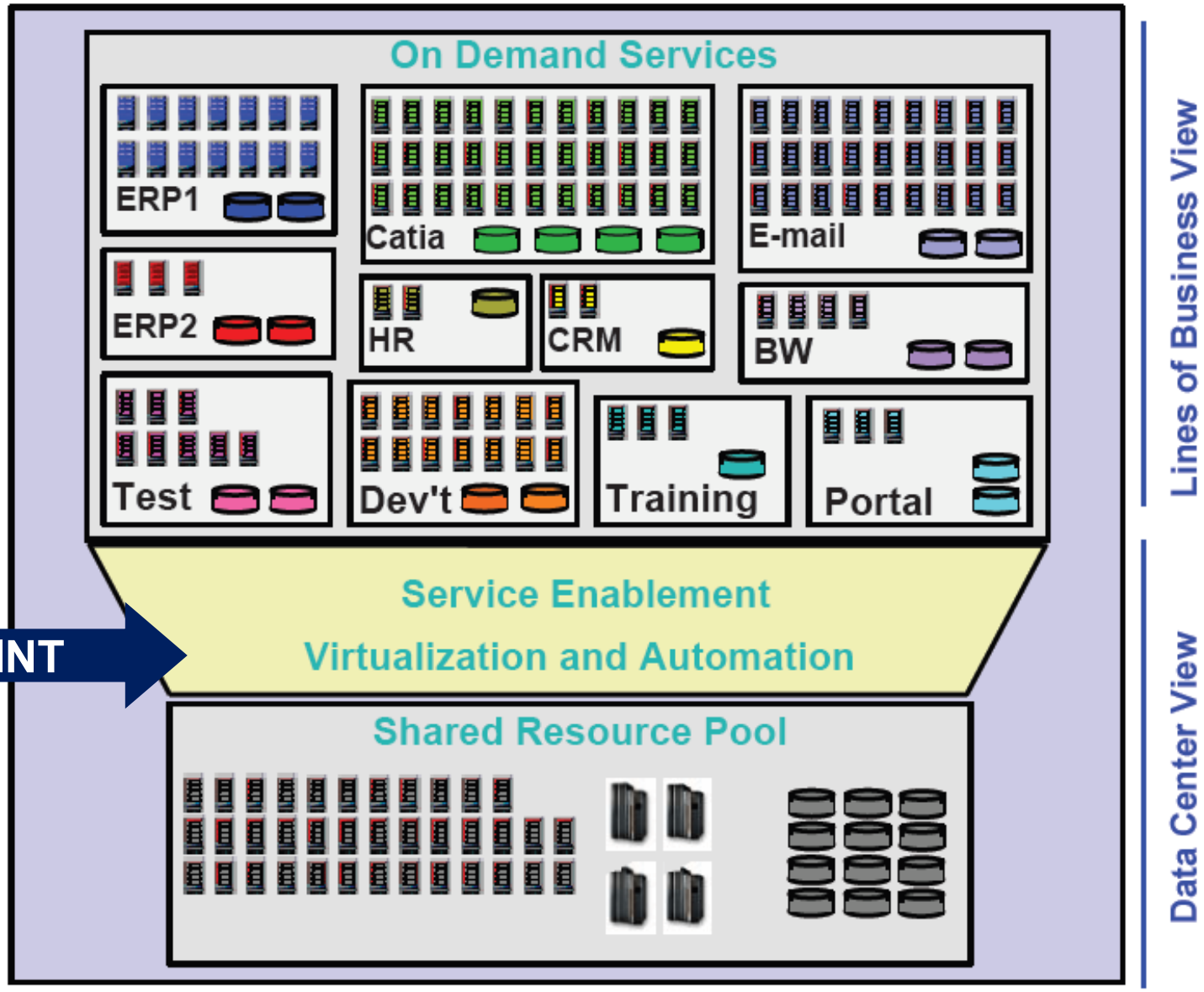


Server replication

Example of a typical distributed infrastructure

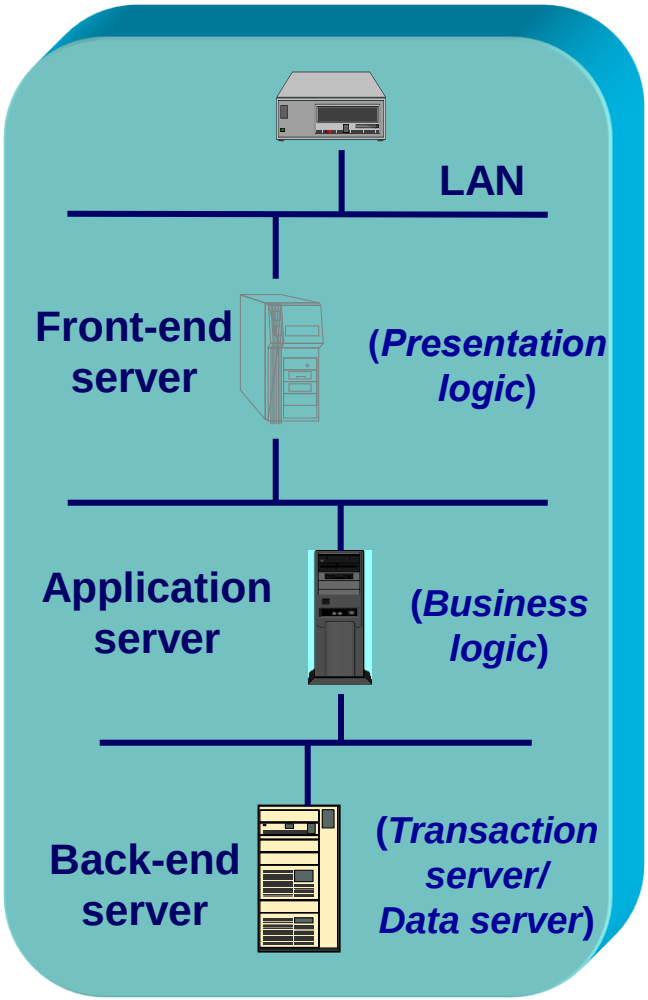


The vision of a modern infrastructure (KISS)

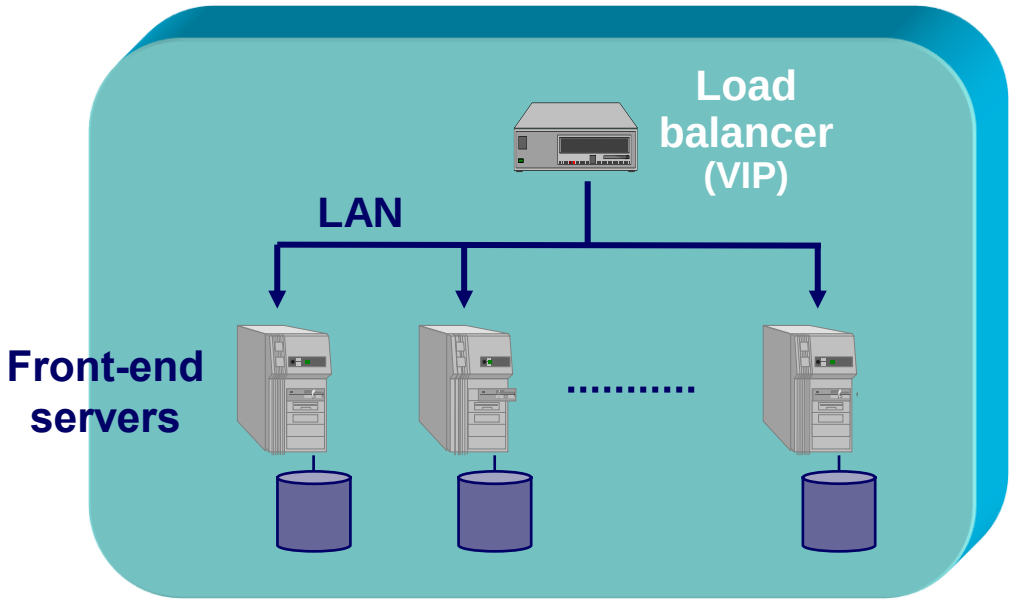


Key POINT

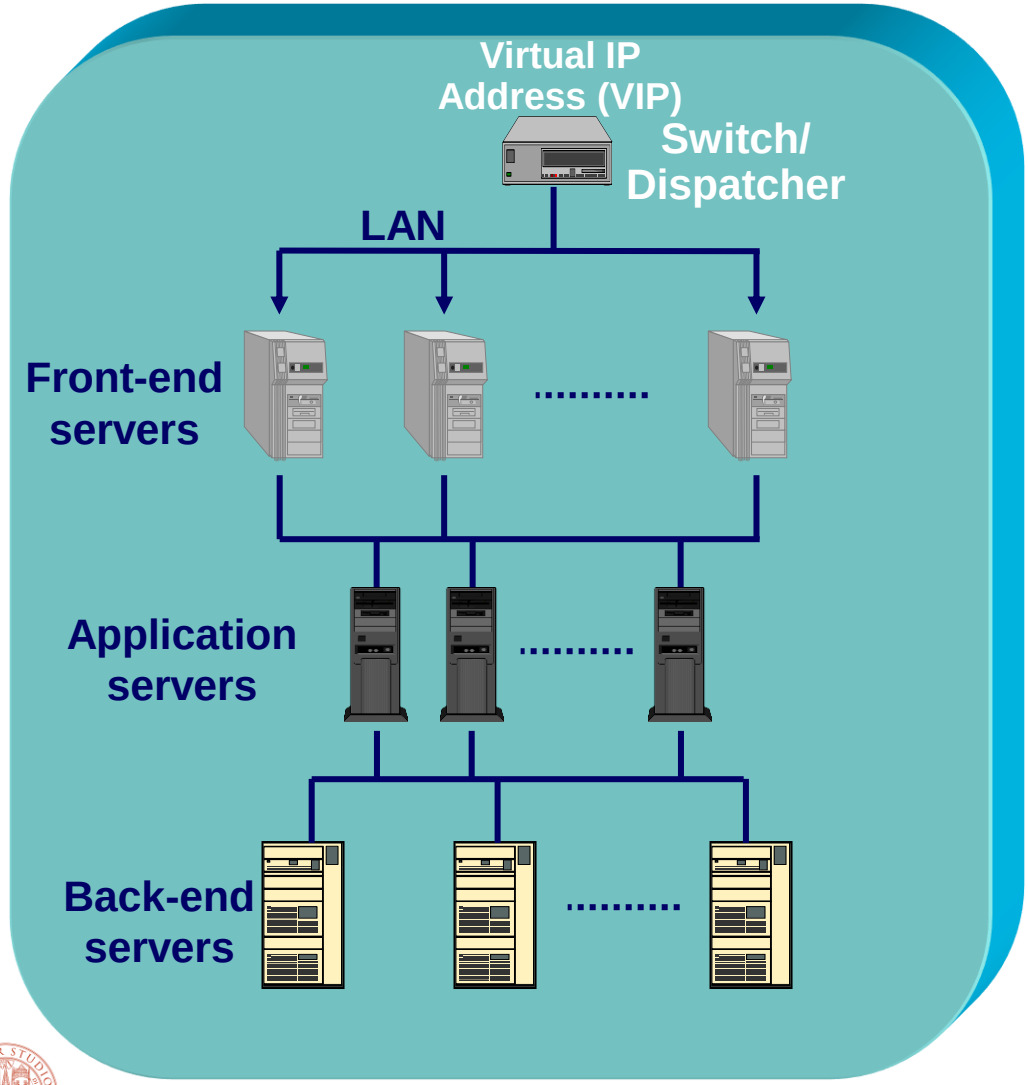
Vertical replication



Horizontal replication



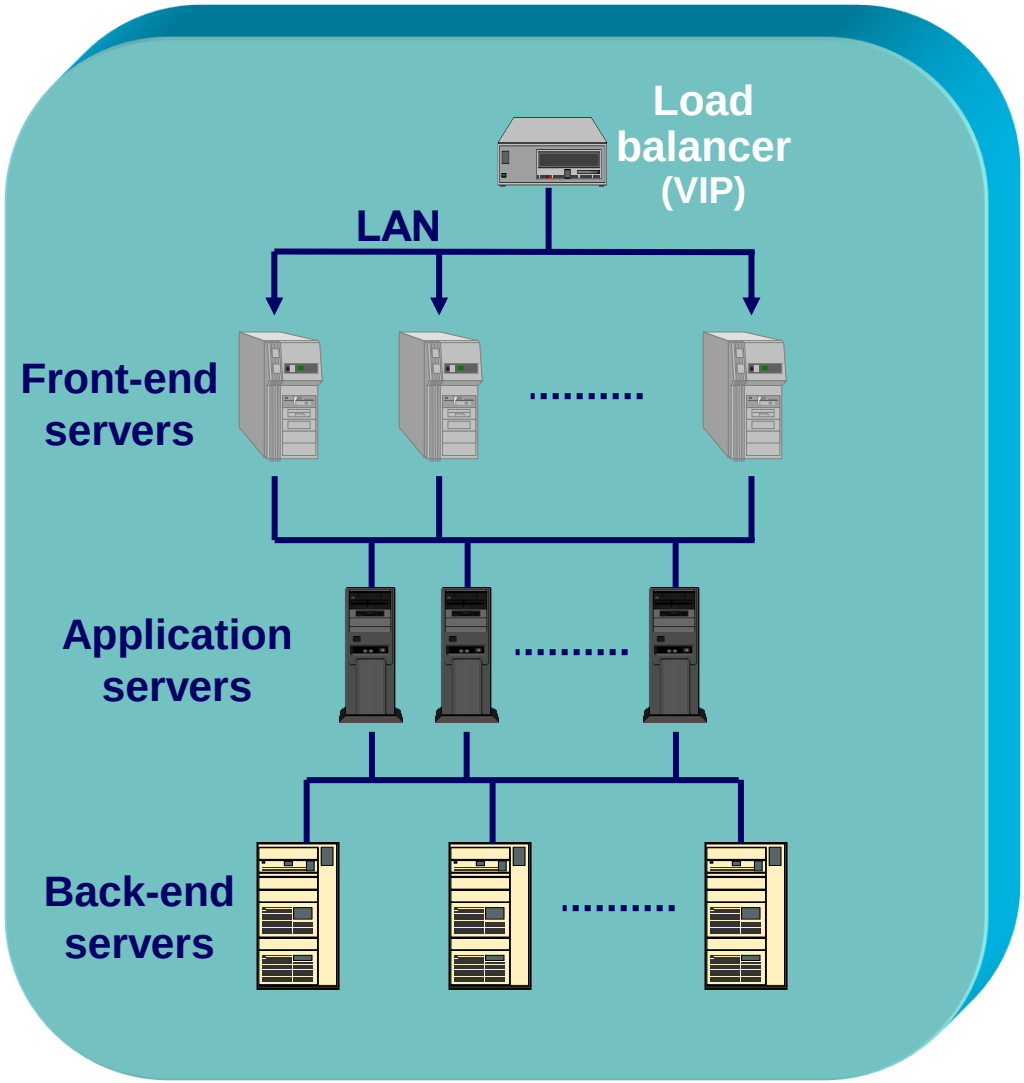
Combinations of Vertical+Horizontal replication



Most *Web-based services* are based on a logical aggregation of virtual machines in one or multiple Data center(s)

The first question for design

1) How many servers can we add?



It depends on the logical level

- **Presentation level**: No conceptual and no implementation limit because user requests are independent
  - Apache
  - Microsoft IIS
- **Application level**: Some limits can reside in the applications and mainly in their implementation
- **Data level**: Severe limits if there are many transactions (“write” operations). Few limits if there is a majority of “read” operations
  - Microsoft Sql server, IBM DB2, Oracle, MySql, Postgress

NO LIMIT ??
➔ INFINITE POWER ??

Second question for design

1) ***How many servers can we add?***

2) **How many servers is it convenient to add?**